

PROJECT DOCUMENTATION

Transaction Management System

Setup, configuration, API reference, and testing guide for the TMS full-stack application (Spring Boot + React).

Generated: 2026-08-09

Source: docs/TMS-Documentation.md

Transaction Management System (TMS)

Project Documentation

A full-stack web application for viewing and managing financial transaction records. The backend is built with **Java 17 / Spring Boot 3** and stores data in a **CSV file**. The frontend is built with **React 18** and served by the **Vite** development server.

This document is the authoritative reference for setting up, configuring, running, and testing the application, and for integrating with its REST API. It reflects the code as of the current `main` state, including pagination/filtering, full CRUD, and the Dashboard/Customers pages.

Table of Contents

1. [Prerequisites](#)
 2. [Installation](#)
 3. [Configuration](#)
 4. [Running the Application](#)
 5. [API Documentation](#)
 6. [Testing](#)
-

1. Prerequisites

Install all of the following before you begin.

Software	Version Required	Used For	Download
Java (JDK)	17 or higher	Runs the Spring Boot backend	https://adoptium.net
Apache Maven	3.6 or higher	Builds and runs the Java backend	https://maven.apache.org/download.cgi
Node.js	18 or higher	Runs the React frontend	https://nodejs.org
npm	9 or higher (bundled with Node.js)	Installs frontend packages	Included with Node.js

No database, message queue, or other external service is required — all data is stored in a CSV file on disk.

Verifying your installations

```
java -version
# Expected: openjdk version "17.0.x" ...

mvn -version
# Expected: Apache Maven 3.9.x ...

node -version
# Expected: v18.x.x or higher

npm -version
# Expected: 9.x.x or higher
```

If any command reports "command not found," that tool is not installed or not on your `PATH`.

2. Installation

1. **Get the code.** Clone or copy the repository, then note its absolute path — it's referred to below as `/path/to/transaction-system`.
2. **Install backend dependencies.**

```
cd /path/to/transaction-system/backend
mvn install
```

Maven reads `pom.xml` and downloads Spring Boot, Bean Validation, and Apache Commons CSV the first time it runs; this can take a few minutes. You do not need to run this as a separate step — `mvn spring-boot:run` (see [Running the Application](#)) will trigger the same download automatically on first use.

3. **Install frontend dependencies.**

```
cd /path/to/transaction-system/frontend
npm install
```

This installs React, Vite, and the Vite React plugin per `package.json`.

No build step is required before running in development mode — `mvn spring-boot:run` and `npm run dev` compile/bundle on the fly.

3. Configuration

Backend — `backend/src/main/resources/application.properties`

```
# The port the backend server listens on
server.port=8080

# Path to the CSV file where transactions are stored (relative to backend/ when run
via Maven)
tms.csv.file-path=src/main/resources/data/transactions.csv
```

Defaults work out of the box. If port `8080` is already in use, change `server.port` and update the frontend proxy target below to match.

CORS

`TransactionController` is annotated with `@CrossOrigin(origins = "http://localhost:5173")` — only that origin may call the API directly from browser JavaScript. This is independent of the Vite proxy (next section); if you change the frontend's dev port, update this annotation too.

Frontend — `frontend/vite.config.js`

```
server: {
  port: 5173,
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true,
    },
  },
},
```

The dev server forwards any request starting with `/api` to the backend, so the frontend never needs a hardcoded API base URL or CORS workaround during development.

Environment variables

None. There are no `.env` files anywhere in the project — all configuration lives in `application.properties` (backend) and `vite.config.js` (frontend).

4. Running the Application

Run the backend and frontend as two separate processes, each in its own terminal.

Start the backend

```
cd /path/to/transaction-system/backend
mvn spring-boot:run
```

Wait for a line like:

```
Started TmsApplication in 2.345 seconds (process running for 2.678)
```

The API is now listening on `http://localhost:8080` . Keep this terminal open.

Start the frontend

```
cd /path/to/transaction-system/frontend
npm install # first run only, or after dependency changes
npm run dev
```

Wait for:

```
VITE v5.x.x ready in 300 ms
→ Local: http://localhost:5173/
```

Open `http://localhost:5173` in a browser. Keep this terminal open as well.

Production frontend build (optional)

```
npm run build # bundles to frontend/dist
npm run preview # serves the production build locally for a sanity check
```

The backend has no equivalent "production" mode distinction for this project — `mvn spring-boot:run` is used the same way in both development and casual production use, or you can package it with `mvn package` and run the resulting jar with `java -jar target/*.jar` .

5. API Documentation

Base URL: `http://localhost:8080` . All endpoints consume and produce JSON. Every `Transaction` returned by the API includes a server-generated `id` (UUID string) — never supplied by the client.

Transaction object

Field	Type	Notes
<code>id</code>	string	Server-generated UUID. Read-only.
<code>transactionDate</code>	string (yyyy-MM-dd)	Required on write.
<code>accountNumber</code>	string	Required, non-blank.
<code>accountHolderName</code>	string	Required, non-blank.
<code>amount</code>	number	Required, must be <code>> 0</code> .
<code>status</code>	string	One of <code>Pending</code> , <code>Settled</code> , <code>Failed</code> . Assigned randomly by the server at creation; read-only thereafter.

GET /api/transactions

Returns a paginated, optionally filtered page of transactions.

Query parameters (all optional, combinable):

Parameter	Type	Default	Description
<code>page</code>	integer	<code>0</code>	Zero-based page index
<code>size</code>	integer	<code>20</code>	Records per page, <code>1 – 100</code>
<code>status</code>	string	—	Exact match: <code>Pending</code> , <code>Settled</code> , <code>Failed</code>
<code>accountNumber</code>	string	—	Exact match
<code>from</code>	date yyyy-MM-dd	—	Inclusive lower bound on <code>transactionDate</code>
<code>to</code>	date yyyy-MM-dd	—	Inclusive upper bound on <code>transactionDate</code>

`totalItems` / `totalPages` reflect the filtered result set, not the whole file.

```
curl "http://localhost:8080/api/transactions?status=Pending&from=2025-01-01&to=2025-06-30&page=0&size=10"
```

200 OK

```
{
  "data": [
    {
      "id": "3a1f9e2c-8b7d-4e11-9c2a-6f0d1b2a3c4d",
      "transactionDate": "2025-03-01",
      "accountNumber": "7289-3445-1121",
      "accountHolderName": "Maria Johnson",
      "amount": 150.0,
      "status": "Settled"
    }
  ],
  "page": 0,
  "size": 10,
  "totalItems": 1,
  "totalPages": 1
}
```

400 Bad Request — invalid `page` / `size` / `status`, or `from` after `to`. **500 Internal Server Error** — CSV file could not be read. Both use the [Error Response Format](#).

POST /api/transactions

Creates a transaction. The server generates `id` and randomly assigns `status`.

```
curl -X POST http://localhost:8080/api/transactions \
  -H "Content-Type: application/json" \
  -d '{"transactionDate":"2026-08-05","accountNumber":"1234-5678-9012","accountHolderName":"Jane Doe","amount":250.00}'
```

201 Created

```
{
  "id": "b6e2c1a0-4d3f-4a2b-9e1c-7f8a9b0c1d2e",
  "transactionDate": "2026-08-05",
  "accountNumber": "1234-5678-9012",
  "accountHolderName": "Jane Doe",
  "amount": 250.0,
  "status": "Settled"
}
```

400 Bad Request — missing/invalid field or malformed JSON. **500 Internal Server Error** — CSV file could not be written.

PUT /api/transactions/{id}

Updates `transactionDate`, `accountNumber`, `accountHolderName`, and `amount` of an existing transaction. `id` and `status` are preserved from the stored record regardless of what's sent.

```
curl -X PUT http://localhost:8080/api/transactions/b6e2c1a0-4d3f-4a2b-9e1c-7f8a9b0c1d2e \
  -H "Content-Type: application/json" \
  -d '{"transactionDate":"2026-08-06","accountNumber":"1234-5678-9012","accountHolderName":"Jane A. Doe","amount":275.00}'
```

200 OK — the updated transaction. **404 Not Found** — no transaction with that `id`. **400 Bad Request** — same validation as `POST`.

DELETE /api/transactions/{id}

```
curl -X DELETE http://localhost:8080/api/transactions/b6e2c1a0-4d3f-4a2b-9e1c-7f8a9b0c1d2e
```

204 No Content — empty body. **404 Not Found** — no transaction with that `id`.

Error Response Format

Every non-2xx response from any endpoint above uses:

```
{ "status": 400, "errors": ["<message>", "..."] }
```

Single-message example:

```
{ "status": 400, "errors": ["page must be >= 0"] }
```

Multi-message example (Bean Validation on the request body):


```
{
  "status": 400,
  "errors": [
    "accountHolderName: Account holder name is required",
    "amount: Amount must be greater than 0"
  ]
}
```

Not backed by a dedicated endpoint

The **Dashboard** and **Customers** pages in the frontend are computed entirely client-side from `GET /api/transactions` results — there is no `/api/dashboard` or `/api/customers` endpoint. Likewise, CSV/Excel export is generated in the browser (`frontend/src/exportUtils.js`), not by the server.

Known limitation: the frontend calls `GET /api/transactions` without `page / size` parameters, so it only ever displays the API's default first page (up to 20 records). Data beyond that is still stored and reachable via the API directly, just not shown in the current UI.

6. Testing

There is currently no automated test suite: `backend/pom.xml` includes `spring-boot-starter-test` but no test classes exist, and `frontend/package.json` has no test runner configured. Verify functionality manually as described below.

Backend — manual API verification (curl)

```
# Happy path
curl "http://localhost:8080/api/transactions?page=0&size=5"

curl -X POST http://localhost:8080/api/transactions \
  -H "Content-Type: application/json" \
  -d '{"transactionDate":"2026-08-05","accountNumber":"1234-5678-9012","accountHolderName":"Jane Doe","amount":250.00}'
# copy the returned "id" for the next two calls

curl -X PUT http://localhost:8080/api/transactions/<id> \
  -H "Content-Type: application/json" \
  -d '{"transactionDate":"2026-08-06","accountNumber":"1234-5678-9012","accountHolderName":"Jane A. Doe","amount":275.00}'

curl -X DELETE http://localhost:8080/api/transactions/<id>
```

```
# Error paths – each should return the {status, errors} shape with the noted code
curl "http://localhost:8080/api/transactions?page=-1" # 400
curl "http://localhost:8080/api/transactions?size=0" # 400
curl "http://localhost:8080/api/transactions?status=Bogus" # 400
curl "http://localhost:8080/api/transactions?from=2026-01-01&to=2025-01-01" # 400
curl -X POST http://localhost:8080/api/transactions -H "Content-Type: application/json" -d '{} ' # 400
curl -X PUT http://localhost:8080/api/transactions/not-a-real-id \
  -H "Content-Type: application/json" \
  -d '{"transactionDate":"2026-08-06","accountNumber":"1","accountHolderName":"X","amount":1}' # 404
curl -X DELETE http://localhost:8080/api/transactions/not-a-real-id # 404
```

Frontend — manual UI checklist

With both servers running, open `http://localhost:5173` :

- **Transactions** — table loads on open; add, edit, and delete a transaction and confirm the row updates immediately; sort each column; filter by status; search by name, account number, and amount; export CSV and `.xlsx` and open the downloaded files to confirm they're valid.
- **Dashboard** — stat cards, monthly volume chart, and status donut reflect current data correctly; "View all transactions" navigates to the Transactions page.
- **Customers** — the customer list matches the distinct account numbers present in your data; search and sorting work; opening a row's drawer shows correct per-customer totals and

transaction history.

- **Error handling** — stop the backend, reload the frontend, and confirm it shows a fetch-error state instead of crashing or hanging silently.

Adding automated tests (future work)

If/when automated coverage is added: backend tests would live under

`backend/src/test/java/com/example/tms/` using the already-declared `spring-boot-starter-test` dependency (JUnit 5 + MockMvc), run via `mvn test` ; frontend tests would need a runner added to `frontend/package.json` (e.g. Vitest) with a corresponding `npm test` script, as none exists today.